(need to just run it) [0x37]

SWAR 0-byte detection

ie marks first bit of 0-bytes & places 0x80 in all 0-bytes, resp

```
mov     rdx, rax
not     rdx
mov     rcx, 0x8080808080808080
and     rdx, rcx
mov     rcx, 0x0101010101010101
sub     rax, rcx
and     rax, rdx
```

$rdx = rax_0$

$rdx = -rax_0 + 1$

$rdx = (-rax_0 + 1) \wedge rcx$

presets only high bit

-1 from every byte piece

subtracts

$rdx = (-rax_0 + 1) \wedge$ (1000(0)1000(0)1000(0)1000(0)1000(0)1000(0)1000(0)1000(0))

$\wedge$

$rax = rax_0 - (0)0001(0)0001(0)0001(0)0001(0)0001(0)0001(0)0001(0)0001$

$\neg x$

$\}$ ie $rdx = \square 0000(0) \square 0000(0) \square 0000(0) \square 0000(0) \square 0000(0) \square 0000(0) \square 0000(0) \square 0000$

empirical asm runs

0xf000 4444 0000 aaaa

0x00 80 0000 80 80 0000

0x aaaaffff 00000000

0x 0000ffff 80 80 80 80

0x 0000 5555 6666 7777

0x 80 80 0000 0000 0000

0x a000 bb00 cc00 80 dd

00 80 0080 0000 80 00

0x8888 aaaa aaaaaaaa

0x0 . . . . . 00

[0x37]

marks the start of every empty (00) byte w/ 1

ie 8 either

```
mov   rdx, rax     rdx := rax
not   rdx          ie   x ↦ -x+1
mov   rcx, 0x8080 8080 8080 8080 ~ (0x 1000 0000 0000 0000)()()()
and   rdx, rcx
mov   rcx, 0x0101 0101 0101 0101
sub   rax, rcx
and   rax, rdx
```

$\Rightarrow$ rax = 1111 ... 1111 − 0000 0001 0000 0001 ... = 1111 1110 1111 1110 ....

rax = rax − rcx = rax − 0x0101 ....

rax − [(0)000 (0)000][·][ ][ ]

rax = 1111 0 ... 1111

rdx = 0 ... 0

rdx = 0 ... 0

1000 (0) 1000 (0) 1000 (0) 1000 (0) 1000 (0) 1000 (0) 1000 (0) 1000 (0) $\Rightarrow$ rdx

If set, was a $0 \in$ rax

$2^8$ $2^0$ $\Rightarrow 2^{8n}$, $n \in \{0, 1, \dots \frac{64}{8}\}$

... 1111 1110 1111 1110 ... $\Rightarrow$ 1 ... 1 ? ex

0